Software Architecture (2026)
Assignment 2

(50 Points)

The primary objective of this assignment is to evaluate the effectiveness of different AI paradigms in supporting software architecture design. In this assignment, you are required to perform architectural design for a given case study (Hotel Pricing System) using the ADD 3.0 method. You will work in groups of **Three** students.

Each group must first select an AI paradigm (direct LLM interaction / single-agent / multi-agent). Then, the group should choose one model from the provided large language models (gemini-3.1-pro-preview/gpt-5.4/deepseek-v4-pro) and combine it with the selected AI paradigm to conduct the ADD 3.0 process and produce the architectural design.

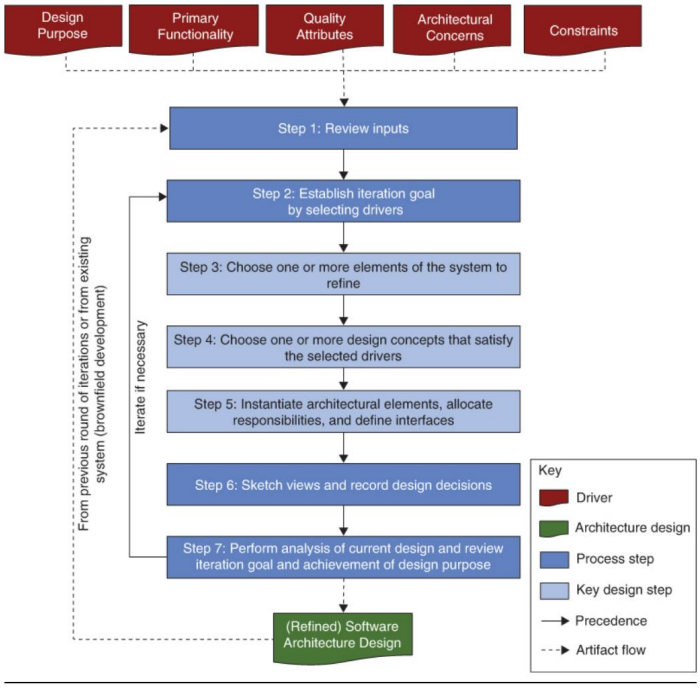

Figure: ADD 3.0 method

DEADLINE: 9:00-11:30 am on (Thursday) June 11, 2026

Submit your group's selection through the Feishu Link (https://my.feishu.cn/sheets/Nj7VsdPfGhh4mWtRVDycQCC1nge?from=from_copylink). The selection of large language models will follow a first-come, first-served principle to avoid duplication. A maximum of **8 groups** may select **the same way** of completing the assignment and **the basic LLM**. The instructor will coordinate adjustments (e.g., through negotiation or lottery) to ensure that for every 8 groups, the chosen way of completing the assignment and the basic LLM are unique. The deadline of the LLM selection is **12:00 am on (Wednesday) June 10, 2026.**

After the team is formed, **please promptly learn how to use Spring AI Alibaba to build single-agent and multi-agent systems.**

Reference link:

1.https://github.com/alibaba/spring-ai-alibaba/tree/main

2.https://github.com/alibaba/spring-ai-alibaba/tree/main/examples

Once your team has been formed, the team leader should complete the registration form via the link below. The teaching assistant will use the submitted information to create API accounts and distribute them during class. These accounts will provide the resources needed to complete the assignment. The accounts are provided solely for assignment execution and resource access; they must not be used for code generation or code-writing assistance.
https://my.feishu.cn/share/base/form/shrcnOxryrbGVEJIKMkVqEShcVO

This assignment is an in-class assignment. Please complete it during class and **submit it to Moodle before the end of the class.**

Assignment Details and Deliverables:

i. Choose the way to complete the assignment (select one of AI paradigms, select one of LLMs (gemini-3.1-pro-preview/gpt-5.4/deepseek-v4-pro))

Option 1. Direct LLM interaction (Implicit single-step reasoning) **:** Carry out the interaction directly using LLM, and use ADD 3.0 to complete the architecture design of the case.

Prior knowledge that must be provided to the AI:

- **ADD 3.0 (see ii. The shared content)**
- **case study-Hotel Pricing System (see ii. The shared content)**
- **Iteration Plan:**

  Iteration 1: Establishing an Overall System Structure

  Iteration 2: Identifying Structures to Support Primary Functionality

  Iteration 3: Addressing Reliability and Availability Quality Attributes

  Iteration 4: Addressing Development and Operations

◆ **Requirements:**

1. Views produced during the iteration should be generated using Mermaid or PlantUML code.

2. No external domain knowledge beyond the provided Prior knowledge is allowed.

3. No few-shot examples or handcrafted demonstration outputs are allowed in the prompt.

4. No additional task reinterpretation or requirement augmentation is allowed beyond the unified Prior knowledge.

5. All decision rules must be explicitly derived from the provided system instructions; no implicit or external rule bases are permitted.

✓ **The key points of writing system instructions:**

1. Prior knowledge

2. Role Prompt

Deliverables include:

1. Source code (15 points)

2. Complete conversation log of the interaction with the LLM (with timestamps included) of four iterations (15 points)

3. Report (see Appendix) (20 points)

Option 2. Single-agent (Sequential reasoning + self-reflection) **:** Create a single-agent and use ADD 3.0 to complete the architecture design of the case.

Prior knowledge that must be provided to the AI:

- **ADD 3.0 (see ii. The shared content)**

- **case study-Hotel Pricing System (see ii. The shared content)**
- **Iteration Plan：**

  Iteration 1: Establishing an Overall System Structure

  Iteration 2: Identifying Structures to Support Primary Functionality

  Iteration 3: Addressing Reliability and Availability Quality Attributes

  Iteration 4: Addressing Development and Operations

◆ **Requirements:**

1. Views produced during the iteration should be generated using Mermaid or PlantUML code.

2. No external domain knowledge beyond the provided Prior knowledge is allowed.

3. No few-shot examples or handcrafted demonstration outputs are allowed in the prompt.

4. No additional task reinterpretation or requirement augmentation is allowed beyond the unified Prior knowledge.

5. All decision rules must be explicitly derived from the provided system instructions; no implicit or external rule bases are permitted.

6. Agents are allowed to decompose tasks, plan reasoning steps, and perform self-verification, as long as these behaviors are derived from system instructions and do not introduce external knowledge.

✓ **The key points of writing system instructions:**

1. Prior knowledge

2. Role Prompt

3. Self-reflection

Deliverables include:

1. Source code (15 points)

2. Complete conversation log of the interaction with the LLM (with timestamps included) of four iterations (15 points)

3. Report (see Appendix) (20 points)

Option 3. Multi-agent（Distributed reasoning + collaborative verification）**:** Create a multi-agent and use ADD 3.0 to complete the architecture design of the case.

Prior knowledge that must be provided to the AI：

- **ADD 3.0 (see ii. The shared content)**
- **case study-Hotel Pricing System (see ii. The shared content)**
- **Iteration Plan：**

  Iteration 1: Establishing an Overall System Structure

  Iteration 2: Identifying Structures to Support Primary Functionality

  Iteration 3: Addressing Reliability and Availability Quality Attributes

  Iteration 4: Addressing Development and Operations

◆ **Requirements:**

1. Views produced during the iteration should be generated using Mermaid or PlantUML code.

2. No external domain knowledge beyond the provided Prior knowledge is allowed.

3. No few-shot examples or handcrafted demonstration outputs are allowed in the prompt.

4. No additional task reinterpretation or requirement augmentation is allowed beyond the unified Prior knowledge.

5. All decision rules must be explicitly derived from the provided system instructions; no implicit or external rule bases are permitted.

6. Agents are allowed to decompose tasks, plan reasoning steps, and perform self-verification, as long as these behaviors are derived from system instructions and do not introduce external knowledge.

✓ **The key points of writing system instructions:**

1. Prior knowledge

2. Multiple Role Prompts

3. Dialogue rules

4. Workflow

Deliverables include:

1. Source code (15 points)

2. Complete conversation log of the interaction with the LLM (with timestamps included) of four iterations (15 points)

3. Report (see Appendix) (20 points)

Notes:

1. You must compile and submityour report in English. You are not necessarily 100% correct in language when using English. However, you should make sure you convey everything in a clear and understandable manner.

2. There is no minimum length requirement for this assignment, but the report should not exceed THIRTYpages (A4 size). *Be concise and to the point.*

3. You can use UML, or other appropriate notations for diagrams. Clearly explain any non- standard symbols used.

ii. The shared content

#Attribute-Driven Design (ADD) method:

##Step 1 **Review Inputs**: The first step of the ADD method involves reviewing the inputs and identifying which requirements will be considered as architectural drivers.

##Step 2 **Establish the Iteration Goal by Selecting Drivers**: A design round generally takes the form of a series of design iterations, where each iteration focuses on achieving a particular goal. Such a goal typically involves designing to satisfy a subset of the drivers.

##Step 3 **Choose One or More Elements of the System to Refine**: This step is where the core design activities start. The elements that you will select are the ones that are involved in the satisfaction of specific drivers. For greenfield development, you can start by establishing the system context and then selecting the only available element—that is, the system itself—for refinement by decomposition. For existing systems or for later design iterations in greenfield systems, you would normally choose to refine elements that were identified in prior iterations.

##Step 4 **Choose One or More Design Concepts That Satisfy the Selected Drivers**: This step requires you to identify alternatives among design concepts that can be used to achieve your iteration goal, and to select one of these alternatives.

##Step 5 **Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces**: This step requires instantiating architectural elements based on the selected design concepts and assigning

 # Use ADD to design a greenfield system in a mature domain: Hotel Pricing System
##Design purpose: This project can be considered greenfield development, as it involves the complete replacement of an existing system. The purpose of the design activity is to make initial decisions to support the construction of the system from scratch.

##Primary Functionality:
|Use Case| Description|
| ---- | ---- |
|HPS-1: Log In| A user (commercial or administrator) provides their credentials in a login window. The system checks these credentials against a user identity service and, if successful, provides access to the system. Once logged in, a user can only make queries and changes to the hotels for which they have been authorized. |
|HPS-2: Change Prices| A user selects a specific hotel for which they are authorized to change prices, and selects dates where they want to make price changes to either a base rate or a fixed rate. All of the prices for the rates that are calculated from the base rate are calculated at that point. The system allows price changes to be simulated before they are actually changed. When the prices are changed, they are pushed to the Channel Management System and become available for querying by external systems. |
|HPS-3: Query Prices| A user or an external system queries prices for a given hotel through the user interface or a query API. |
|HPS-4: Manage Hotels| An administrator adds, changes, or modifies hotel information. This includes editing the hotel's tax rates, available rates, and room types. |
|HPS-5: Manage Rates| An administrator adds, changes, or modifies rates. This includes defining the calculation business rules for the different rates. |
|HPS-6: Manage Users| An administrator changes permissions for a given user. |

##Quality Attributes:
|ID |Quality Attribute |Scenario |Associated Use Case| Importance to the Customer |Difficulty of Implementation |
| ---- | ---- | ---- |---- | ---- |---- |
|QA-1|Performance|A base rate price is changed for a specific hotel and date during normal operation; the prices for all the rates and room types for the hotel are published (ready for query) in less than 100 ms. |HPS-2|High|High|
|QA-2|Reliability|A user performs multiple price changes on a given hotel; 100% of the price changes are published (available for query) successfully and are also received by the Channel Management System.|HPS-2| High|High|
|QA-3|Availability|Pricing queries uptime SLA must be 99.9% outside of maintenance windows.|All| High|High|
|QA-4|Scalability| The system will initially support a minimum of 100,000 price queries per day through its API and should be capable of handling up to 1,000,000 without decreasing average latency by more than 20%.|HPS-3| High|High|
|QA-5|Security|A user logs into the system through the front-end. The credentials of the user are validated against the User Identity Service and, once logged in, they are presented with only the functions that they are authorized to use.|All| High| Medium|
|QA-6|Modifiability|Support for a price query endpoint with a different protocol than REST (e.g., gRPC) is added to the system. The new endpoint does not require changes to be made to the core components of the system.|All|Medium| Medium|
|QA-7| |Deployability|The application is moved between nonproduction environments as part of the development process. No changes in the code are needed.|All | Medium| Medium|
|QA-8| |Monitorability| A system operator wishes to measure the performance and reliability of price publication during operation. The system provides a mechanism that allows 100% of these measures to be collected as needed.| HPS-2| Medium| Medium|
|QA-9|Testability|100% of the system and its elements should support integration testing independently of the external systems.| All| Medium| Medium|

##Architectural Concerns:
|ID |Concern|
|CRN-1| Establish an overall initial system structure. |
|CRN-2| Leverage the team's knowledge about Java technologies, the Angular framework, and Kafka. |
|CRN-3| Allocate work to members of the development team. |
|CRN-4| Avoid introducing technical debt.|
|CRN-5| Set up a continuous deployment infrastructure.|

##Constraints:

|ID |Constraint|
|CON-1 |Users must interact with the system through a web browser
in different platforms (Windows, OSX, and Linux, and different devices).|
|CON-2 | Manage users through cloud provider identity service and host resources in the cloud.|
|CON-3 | Code must be hosted on a proprietary Git-based platform that is already in use by other projects in the company. |
|CON-4 | The initial release of the system must be delivered in 6 months, but an initial version of the system (MVP) must be demonstrated to internal stakeholders in at most 2 months. |
|CON-5 | The system must interact initially with existing systems through REST APIs but may need to later support other protocols. |
|CON-6 | A cloud-native approach should be favored when designing the system.|

Appendix: The Report Template (*You must compile and submit your report in English. There is no minimum length requirement for this assignment, but the report should not exceed THIRTYpages* (A4 size). *Be concise and to the point.*)

一、 Output results of ADD

ADD Step 1:

1) Output results of each step (Iteration 1: Establishing an Overall System Structure)

ADD Step 2:
ADD Step 3:
ADD Step 4:
ADD Step 5:
ADD Step 6:
ADD Step 7:

2) Output results of each step (Iteration 2: Identifying Structures to Support Primary Functionality)

ADD Step 2:
ADD Step 3:
ADD Step 4:
ADD Step 5:
ADD Step 6:
ADD Step 7:

3) Output results of each step (Iteration 3: Addressing Reliability and Availability Quality Attributes)

ADD Step 2:
ADD Step 3:
ADD Step 4:
ADD Step 5:
ADD Step 6:
ADD Step 7:

4) Output results of each step (Iteration 4: Addressing Development and Operations)

ADD Step 2:
ADD Step 3:
ADD Step 4:
ADD Step 5:
ADD Step 6:
ADD Step 7:

二、 Interaction cost analysis

The way of completing the assignment	
The LLM used	
Number of Human Interactions (turns)	
Token Consumption (K tokens)	

Time Cost （min）	

三、 Individual Reflection

1) The problems encountered and the solutions adopted

2) A detailed account of your personal contributions to the group work

Name（Chinese）	Contributions